



It's Your Life

with





Vue

스타일링



클래스와 스타일 바인딩

일반적으로 엘리먼트에 데이터를 바인딩하는 이유는 클래스 목록과 해당 인라인 스타일을 조작하기 위함입니다. `class`, `style` 둘 다 속성이므로 다른 속성과 마찬가지로 `v-bind`를 사용하여 문자열 값을 동적으로 할당할 수 있습니다. 그러나 연결된 문자열을 사용하여 이러한 값을 생성하려고 하면 성가시고 오류가 발생하기 쉽습니다. 이러한 이유로 Vue는 `v-bind`가 `class` 및 `style`과 함께 사용될 때 특별한 향상을 제공합니다. 문자열 외에도 표현식은 객체 또는 배열로 평가될 수 있습니다.



인라인

스타일



인라인 스타일 왜 쓰나요!?

- '인라인' 이 붙으면 이제 감이 오시죠!?
뭔가 급하고 빠르게 그리고 간단하게
무언가를 적용하고 싶을 때 사용합니다!
- 대신 스타일은 여러 개를 넣어야 하
므로 여러 개의 값을 적용하기 위해
서 객체를 사용하여 전달 합니다!

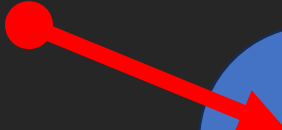


인라인 스타일 사용하기!

일반 스타일

Vue 서타일

```
<body>
  <div id="app">
    <h1 style="background-color: skyblue">일반 스타일</h1>
    <h1 v-bind:style="{ backgroundColor : 'orange' }">Vue 서타일</h1>
  </div>
</body>
<script>
  const vueApp = Vue.createApp({
    name: 'App',
  }).mount('#app');
</script>
```



v-bind 디렉티브를 쓰면
= 뒤의 문자열은 JS 로 처리가 되므로
객체를 넣어 줄 수 있습니다!!

대신 '-' 은 빼기로 인식이 되기 때문에
style 의 kebab-case 를
camelCase 로 변경해 줘야 합니다!



v-bind 디렉티브를 쓰면
= 뒤의 문자열은 JS 로 처리가 되므로
객체를 넣어 줄 수 있습니다!!

그럼 변수도 가능하겠군요!?





```
<body>
  <div id="app">
    <h1 style="background-color: skyblue">일반 스타일</h1>
    <h1 v-bind:style="{ backgroundColor : 'orange' }">Vue 서타일</h1>
    <h1 v-bind:style="variableStyle">Vue 변수 서타일</h1>
  </div>
</body>
<script>
  const vueApp = Vue.createApp({
    name: 'App',
    data() {
      return {
        variableStyle: { backgroundColor: 'dodgerblue' },
      };
    },
  }).mount('#app');
</script>
```

v-bind 를 이용하여
해당 데이터를 스타일로 지정

Vue 데이터(=변수)에
객체 형태로 스타일을 지정

일반 스타일



Vue 서타일

Vue 변수 서타일





인라인 스타일

여러 개 넣기!

```
<body>
  <div id="app">
    <h1 v-bind:style="{backgroundColor, color}">
      여러 서타일 적용하기 by 객체
    </h1>
  </div>
</body>
<script>
  const vueApp = Vue.createApp({
    name: 'App',
    data() {
      return {
        backgroundColor: 'dodgerblue',
        color: 'red',
      };
    },
  }).mount('#app');
</script>
```

객체에 넣어서 사용

스타일에 사용할 데이터를
미리 선언!

단, CSS의 속성명 : 값
형태만 가능합니다!!



객체를 쓰면 객체 내부에는 반드시
css 에 관련된 문법을 따라야만 합니다!



```
<body>
  <div id="app">
    <h1 v-bind:style="{backgroundStyle, fontStyle}">
      여러 서타일 적용하기 by 객체
    </h1>
    <h1 v-bind:style="[backgroundStyle, fontStyle]">
      여러 서타일 적용하기 by 배열
    </h1>
  </div>
</body>
<script>
  const vueApp = Vue.createApp({
    name: 'App',
    data() {
      return {
        backgroundStyle: { backgroundColor: 'dodgerblue' },
        fontStyle: { color: 'red' },
      };
    },
  }).mount('#app');
</script>
```

객체에 넣어서 사용

배열에 넣어서 사용

스타일에 사용할 데이터를
미리 선언!

여러 서타일 적용하기 by 객체

여러 서타일 적용하기 by 배열





```
<h1 v-bind:style="{backgroundStyle, fontStyle}">  
  여러 서타일 적용하기 by 객체  
</h1>
```

```
style = {  
  backgroundColor: 'dodgerblue',  
  color: 'red',  
};
```

우리가 원했던 것

```
style = {  
  backgroundStyle: {  
    backgroundColor: 'dodgerblue',  
  },  
  fontStyle: {  
    color: 'red',  
  },  
};
```

실제로 적용 된 것



★상상과 실제 GAP차이★



내가 생각한 20대

현실 20대



이상 VS 현실


```
<h1 v-bind:style="{...backgroundStyle, ...fontStyle}">  
  여러 서타일 적용하기 by 객체  
</h1>
```



객체가 중첩되어 문제가 일어나는 것을
피하기 위해서 전개 연산자를 사용하여 뿌려 줍니다!

```
style = {  
  backgroundStyle: {  
    backgroundColor: 'dodgerblue',  
  },  
  fontStyle: {  
    color: 'red',  
  },  
};
```





굳이... 할 필요가



실습, 인라인 스타일 적용하기!

- Vue 의 데이터에 아래의 조건을 만족하는 변수 bgOrange, bgSkyblue, fontRed, fontWhite 를 선언하세요.
- bgOrange : 배경을 오렌지색으로 변경
- bgSkyblue : 배경을 하늘색으로 변경
- fontRed : 글자색을 빨간색으로 변경
- fontWhite : 글자색을 하얀색으로 변경



실습, 인라인 스타일 적용하기!

```
<body>
  <div id="app">
    <h1>배경은 하늘색이고, 글자는 빨간색</h1>
    <h1>배경은 오렌지색이고, 글자는 하얀색</h1>
  </div>
</body>
```

- 인라인 스타일을 사용하여, h1 태그의 내용과 스타일을 맞춰 주세요



클래스

바인딩



클래스 바인딩 왜 쓰나요!?

- css 를 해보셔서 아시겠지만 css 로 웹페이지를 예쁘게(?) 만들려면 많은 속성 값들을 한꺼번에 지정해 줘야 합니다.
- 그럴 때 마다 인라인 스타일을 쓴다면!?
- 혹은 이미 잘 만들어진 css 스타일이 존재 한다면!?



복동아! 지금이야!



절대

클래스

쓰시는 편이 편합니다!


```
<style>
  .bg-orange {
    background-color: orange;
  }
  .font-white {
    color: white;
  }
</style>
</head>
<body>
  <div id="app">
    <h1 v-bind:class="myStyle">배경은 하늘색</h1>
  </div>
  <script>
    const vueApp = Vue.createApp({
      name: 'App',
      data() {
        return {
          myStyle: 'bg-orange font-white',
        };
      },
    }).mount('#app');
  </script>
</body>
```

미리 선언 되어 있는
css 속성들

v-bind:class 를 이용하여
myStyle 데이터를 삽입
→ bgOrange fontWhite 가 적용

클래스 명을 문자열로 전달



배경은 하늘색

```
▼<div id="app" data-v-app>  
  <h1 class="bg-orange font-white">배경은 하늘색</h1> =  
</div>  
▶<script>👾</script>
```



동적

Vue 스타일링

그럼 동적으로 스타일을 바꾸고 싶을때!?



- 버튼을 클릭하면 태그에 적용하는 클래스의 이름을 바꾸고, 해당 클래스에 따라 배경색을 바꾸는 웹 페이지를 만들어 봅시다

```
<style>
  .bg-orange {
    background-color: orange;
  }
  .bg-skyblue {
    background-color: skyblue;
  }
</style>
</head>
<body>
  <div id="app">
    <h1 v-bind:class="myStyle">배경은?</h1>
    <button v-on:click="toggleStyle">배경 토글</button>
  </div>
```

미리 사용할 css 속성을 선언

동적으로 변경할 클래스를 위한
computed 객체 바인딩

동적으로 값을 변경하는 메소드를
v-on:click 이벤트로 연결



```
<script>
  const vueApp = Vue.createApp({
    name: 'App',
    data() {
      return {
        isOrange: true,
      };
    },
    methods: {
      toggleStyle() {
        this.isOrange = !this.isOrange;
      },
    },
    computed: {
      myStyle() {
        return this.isOrange ? 'bg-orange' :
      },
    },
  }).mount('#app');
</script>
```

동적 스타일의 트리거가 될
데이터 선언

isOrange 데이터의 값을
true/false 로 변경하는 메서드

computed 를 사용하여
isOrange 데이터의 값에 따라
'bg-orange' 또는 'bg-skyblue'를 리턴

배경은 ???

배경 토글

```
▼ <div id="app" data-v-app> == $0  
  <h1 class="bg-orange">배경은?</h1>  
  <button>배경 토글</button>  
  </div>
```

배경은 ???

배경 토글

```
▼ <div id="app" data-v-app> == $0  
  <h1 class="bg-skyblue">배경은?</h1>  
  <button>배경 토글</button>  
  </div>
```





실습, 동적 스타일 연습하기!

- 체크 박스의 체크 여부에 따라서 <h1> 태그의 배경색과 내용을 바꾸는 웹페이지를 만들어 주세요!

배경은 ???

오렌지색 ? ☐



실전 버전!



동적

Vue 스타일링

with Tailwind



팔레트 아이콘을 누를 때 마다
전체 페이지의 theme 가 변경되는
페이지를 만들어 봅시다~!

COUNTER

0

UP

DOWN

<https://github.com/xenosign/kb6-front/blob/main/test/vue/tailwind-counter-theme.html>



 **xenosign** 250324 수업 준비

b30cb6e · 8 hours ago  History

CodeBlame102 lines (100 loc) · 3.45 KB Code 55% faster with GitHub Copilot

RawCopyDownloadEditDropdownCompare

```
1  <!DOCTYPE html>
2  <html lang="en">
3    <head>
4      <meta charset="UTF-8" />
5      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6      <title>Vue Counter with Tailwind</title>
7      <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
8      <script src="https://cdn.tailwindcss.com"></script>
9      <link
10        href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;500;600;700&display=swap"
11        rel="stylesheet"
12      />
13      <link
14        href="https://fonts.googleapis.com/icon?family=Material+Icons"
15        rel="stylesheet"
16      />
17      <style>
18        body {
19          font-family: 'Poppins', sans-serif;
20        }
21      </style>
22    </head>
23    <body>
24      <div id="app">
25        <div class="fixed top-6 right-6 z-50">
```

코드는

여기 있습니다

A cartoon illustration of a person's face with wide, circular eyes and a small, open mouth, indicating surprise or realization. A hand is visible in the bottom left corner, pointing towards the text.

테마를 변경하는 switchBackground
메서드를 클릭 이벤트로 연결

```
<div class="fixed top-6 right-6 z-50">
  <button
    @click="switchBackground"
    class="p-3 rounded-full bg-white bg-opacity-20 backdrop-blur-lg shadow-lg
hover:bg-opacity-40 transition-all duration-300 text-white"
  >
    <span class="material-icons">palette</span>
  </button>
</div>
```



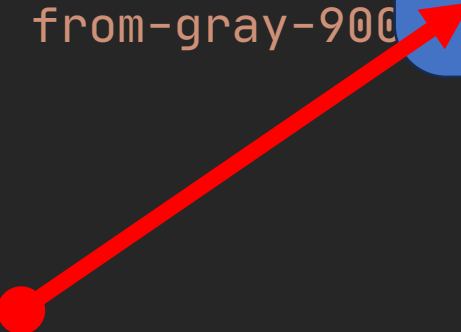
실제 테마를 변경하는
index 용 데이터 선언

배경으로 사용할 theme 의 테일윈드
클래스 명을 미리 배열로 선언하여 사용

```
createApp({
  name: 'App',
  data() {
    return {
      count: 0,
      bgIndex: 0,
      backgrounds: [
        'bg-gradient-to-br from-indigo-900 via-purple-900 to-pink-800',
        'bg-gradient-to-br from-green-900 via-emerald-800 to-teal-700',
        'bg-gradient-to-br from-blue-900 via-cyan-800 to-sky-700',
        'bg-gradient-to-br from-gray-900 via-gray-800 to-gray-700',
      ],
    };
  },
  computed: {
    currentBackground() {
      return this.backgrounds[this.bgIndex];
    },
  },
},
```

```
createApp({
  name: 'App',
  data() {
    return {
      count: 0,
      bgIndex: 0,
      backgrounds: [
        'bg-gradient-to-br from-indigo-900 to-purple-900',
        'bg-gradient-to-br from-green-900 to-teal-900',
        'bg-gradient-to-br from-blue-900 to-indigo-900',
        'bg-gradient-to-br from-gray-900 to-black',
      ],
    };
  },
  computed: {
    currentBackground() {
      return this.backgrounds[this.bgIndex];
    },
  },
},
```

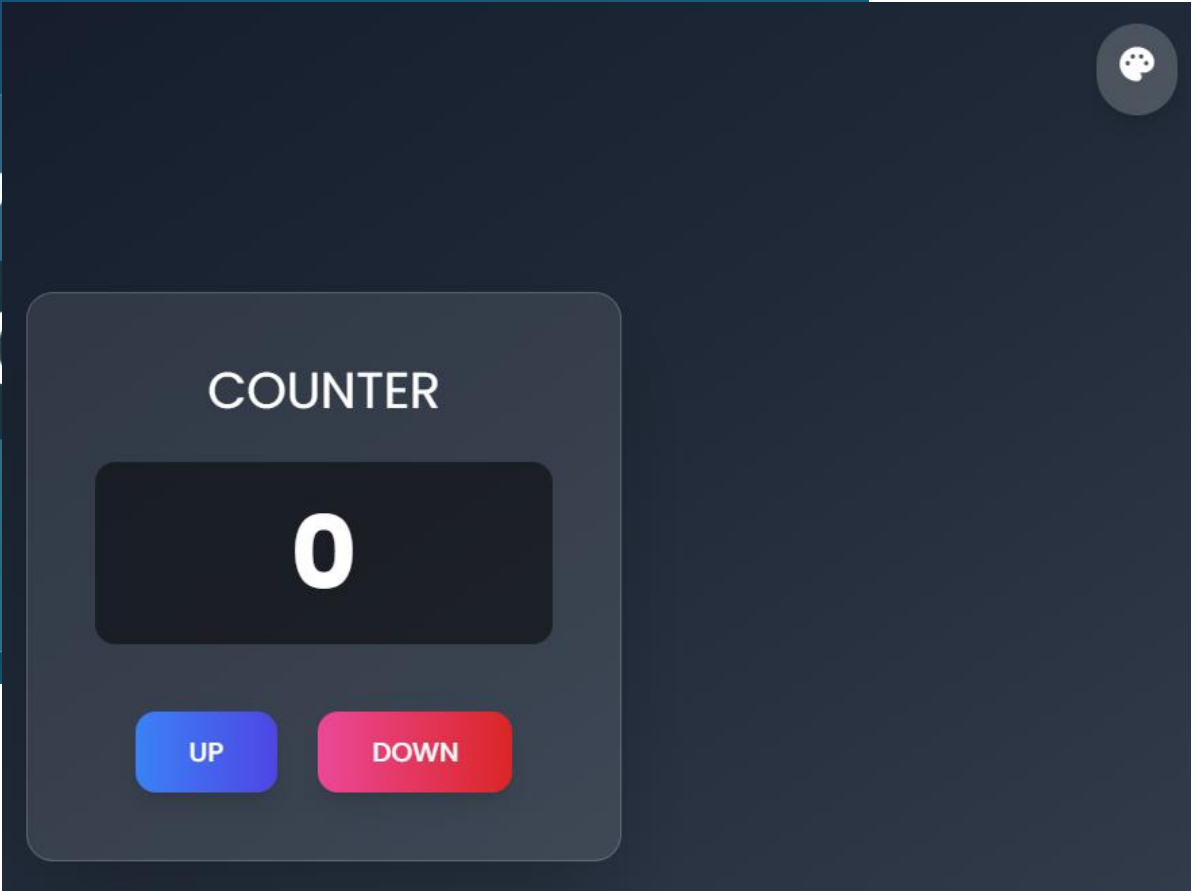
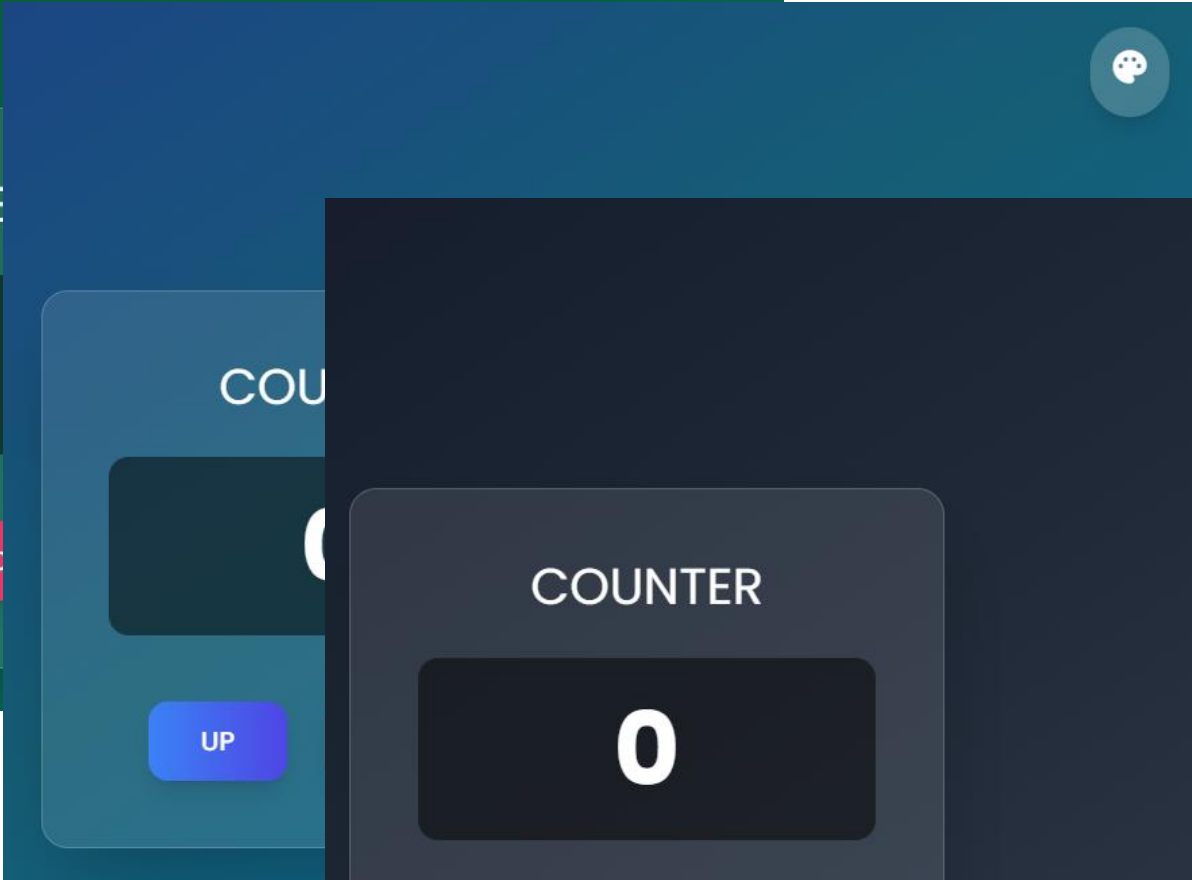
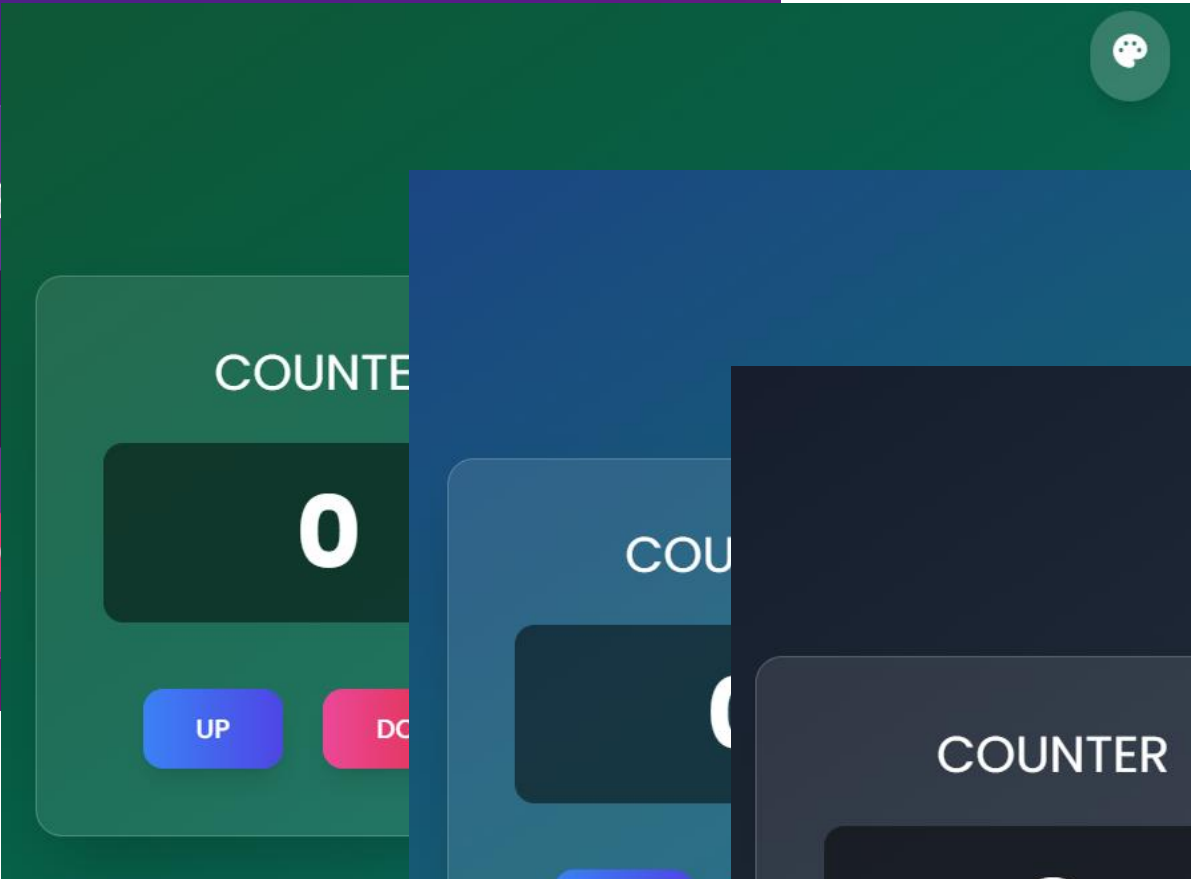
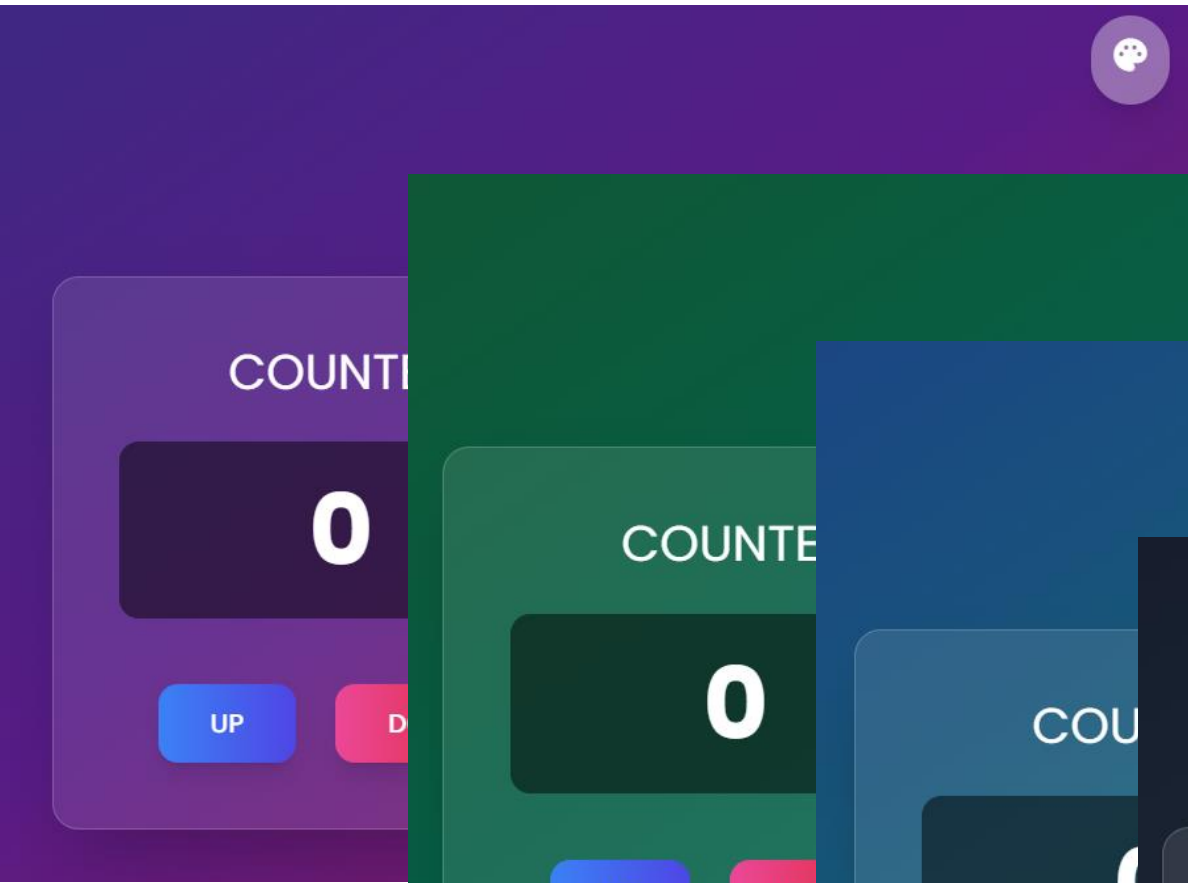
bgIndex 의 값에 따라 다른 theme 가
적용되어야 하므로 computed 를
사용하여 처리



```
methods: {  
  countUp() {  
    this.count++;  
  },  
  countDown() {  
    this.count--;  
  },  
  switchBackground() {  
    this.bgIndex = (this.bgIndex + 1) % this.backgrounds.length;  
    document.body.className = this.currentBackground;  
  },  
},  
mounted() {  
  document.body.className = this.backgrounds[0];  
},  
}
```

팔레트 아이콘을 클릭 될 때 실행되는
메서드로 bgIndex 값을 변경!

DOM 에 대한 디자인 요소를 반영하는 작업이므로
DOM 이 root 에 mounted 된 다음 작업을 처리해야
안정적으로 수행이 가능!





비동기 통신 처리



v-if 로

로딩 Spinner 구현



기억 나시나요!?

Tetz's favorite foods with Computed

카테고리 선택 :

- 제육
- 돈까스
- 광어회
- 탄탄면
- 어항동고
- 마파두부
- 초밥
- 라멘
- 타코야끼



그간 배운 걸
한 번 적용해 봅시다!

장미향 같은 웃음 원하신가요?



```
data() {  
  return {  
    foods: [],  
    selectedCategory: 'all',  
    isLoading: true,  
  };  
},
```

비동기 통신 상태를 관리하는
isLoading 데이터를 만들어 봅시다!



```
<div id="app">
  <div :class="isLoading ? 'loading-bg' : 'success-bg'">
    <h1>Tetz's favorite foods with Computed</h1>
    <label for="category">카테고리 선택 : </label>
    <select id="category" v-model="selectedCategory">
      <option value="all">전체</option>
      <option value="korean">한식</option>
      <option value="japanese">일식</option>
      <option value="chinese">중식</option>
    </select>

    <div class="loading-container" v-if="isLoading">
      <div class="spinner"></div>
      <p>데이터 로딩 중...</p>
    </div>

    <ul v-else>
      <li v-for="(food, index) in filteredFoods" :key="index">
        {{ food.food }}
      </li>
    </ul>
  </div>
</div>
```

v-if 와 isLoading 데이터를 사용
로딩 상태를 보여주는 spinner 추가

로딩이 완료 되면 v-else 에 의해
음식 목록 출력

```
<style>
.spinner {
  width: 40px;
  height: 40px;
  margin: 30px;
  border: 4px solid rgba(0, 0, 0, 0.1);
  border-radius: 50%;
  border-top: 4px solid #3498db;
  animation: spin 1s linear infinite;
}

@keyframes spin {
  0% {
    transform: rotate(0deg);
  }
  100% {
    transform: rotate(360deg);
  }
}
</style>
```

순수 css 와 animation 을 이용하여
spinner 를 구현

사용자 정의 애니메이션을 만드는
@keyframes

0% → 100% 까지 해당
css 속성을 시간에
맞춰 변화 시킵니다

카테고리 선택 : 전체 ▼



데이터 로딩 중...

?)의 운동신경 이규한





isLoading 값에

따른 동적 스타일링!

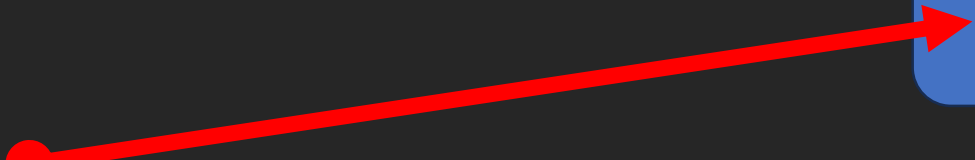
```
.loading-bg {  
  background: linear-gradient(-90deg, #3498db, #9b59b6, #2ecc71, #f1c40f);  
  background-size: 400% 400%;  
  animation: gradient-wave 5s ease infinite;  
}
```

```
@keyframes gradient-wave {  
  0% {  
    background-position: 100% 50%;  
  }  
  50% {  
    background-position: 0% 50%;  
  }  
  100% {  
    background-position: 100% 50%;  
  }  
}
```

```
.success-bg {  
  background-color: orange;  
}
```



로딩 상황에 적용할
css 클래스 선언



로딩이 끝나면 적요할
css 클래스 선언

v-bind 와 data 를 이용
data의 true/false 결과에 따라
각기 다른 class 를 적용!

```
<div id="app">
  <div :class="isLoading ? 'loading-bg' : 'success-bg'">
    <h1>Tetz's favorite foods with Computed</h1>
    <label for="category">카테고리 선택 : </label>
    <select id="category" v-model="selectedCategory">
      <option value="all">전체</option>
      <option value="korean">한식</option>
      <option value="japanese">일식</option>
      <option value="chinese">중식</option>
    </select>
```

비동기 통신을 시작하면
isLoading 을 true 로 변경

```
methods: {  
  async fetchFoods() {  
    this.isLoading = true;  
  
    try {  
      const response = await fetch(  
        'https://port-0-tetz-night-back-m5yo5gmx92cc34bc.sel4.cloudtype.app/food/all'  
      );  
      this.foods = await response.json();  
    } catch (e) {  
      console.error('API 오류 발생:', e);  
    } finally {  
      this.isLoading = false;  
    }  
  },  
},
```

비동기 통신이 끝나면
isLoading 을 false 로 변경하여
작업 완료!



Tetz's favorite foods with Computed

카테고리 선택 : 전체 ▾



로딩 중에는 배경에 loading-bg 이 적용
애니메이션이 & Spinner 작동

데이터 로딩 중...

Tetz's favorite foods with Computed

카테고리 선택 : 전체 ▾

- 제육
- 돈까스
- 광어회
- 탄탄면
- 어항동고
- 마파두부
- 초밥
- 라멘
- 타코야끼



데이터 로딩이 끝나면
배경에 success-bg 적용 & 목록 출력

